

```
1
2 from google.colab import drive
3 drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
1
2 import pandas as pd
3 import numpy as np
4 import seaborn as sns
5 import matplotlib.pyplot as plt
6
```

```
1
2 telcom = pd.read_csv("/content/drive/MyDrive/Telcom Customer Churn Project/WA_Fn-UseC_-Telco-Customer-Churn.csv")
```

```
1
2 telcom.head()
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	...	DeviceProtection	TechSupport	StreamingTV	StreamingMovies
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	No	...	No	No	No	
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes	...	Yes	No	No	
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Yes	...	No	No	No	
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes	...	Yes	Yes	No	
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No	...	No	No	No	

5 rows × 21 columns

```
1
2 # Checking Duplicties
3 telcom.duplicated().sum()
```

np.int64(0)

```
1
2 # Checking null
3 telcom.isnull().sum()
```

	0
customerID	0
gender	0
SeniorCitizen	0
Partner	0
Dependents	0
tenure	0
PhoneService	0
MultipleLines	0
InternetService	0
OnlineSecurity	0
OnlineBackup	0
DeviceProtection	0
TechSupport	0
StreamingTV	0
StreamingMovies	0
Contract	0
PaperlessBilling	0
PaymentMethod	0
MonthlyCharges	0
TotalCharges	0
Churn	0

dtype: int64

```
1
2 # Change columns (like SeniorCitizen -> Senior Citizen)
3 telcom.columns = telcom.columns.str.replace(r'([a-z0-9])([A-Z])', r'\1_2', regex=True).str.title()
4
```

```
1
2 telcom.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Customer_Id           7043 non-null   object
1   Gender                7043 non-null   object
2   Senior_Citizen        7043 non-null   int64
3   Partner               7043 non-null   object
4   Dependents            7043 non-null   object
5   Tenure                7043 non-null   int64
6   Phone_Service         7043 non-null   object
7   Multiple_Lines        7043 non-null   object
8   Internet_Service      7043 non-null   object
9   Online_Security       7043 non-null   object
10  Online_Backup         7043 non-null   object
11  Device_Protection     7043 non-null   object
12  Tech_Support          7043 non-null   object
13  Streaming_Tv          7043 non-null   object
14  Streaming_Movies      7043 non-null   object
```

```
15 Contract          7043 non-null    object
16 Paperless_Billing 7043 non-null    object
17 Payment_Method     7043 non-null    object
18 Monthly_Charges    7043 non-null    float64
19 Total_Charges      7043 non-null    object
20 Churn              7043 non-null    object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

```
1
2 # Checking Unique Values to understand column data and find potential typos
3 for col in telcom.columns:
4     unique_vals = telcom[col].unique()
5     print(f"{col}: {unique_vals}\n\n{'-'*30}")
```

```
-----
Partner: ['Yes' 'No']

-----
Dependents: ['No' 'Yes']

-----
Tenure: [ 1 34  2 45  8 22 10 28 62 13 16 58 49 25 69 52 71 21 12 30 47 72 17 27
  5 46 11 70 63 43 15 60 18 66  9  3 31 50 64 56  7 42 35 48 29 65 38 68
32 55 37 36 41  6  4 33 67 23 57 61 14 20 53 40 59 24 44 19 54 51 26  0
39]

-----
Phone_Service: ['No' 'Yes']

-----
Multiple_Lines: ['No phone service' 'No' 'Yes']

-----
Internet_Service: ['DSL' 'Fiber optic' 'No']

-----
Online_Security: ['No' 'Yes' 'No internet service']

-----
Online_Backup: ['Yes' 'No' 'No internet service']

-----
Device_Protection: ['No' 'Yes' 'No internet service']

-----
Tech_Support: ['No' 'Yes' 'No internet service']

-----
Streaming_Tv: ['No' 'Yes' 'No internet service']

-----
Streaming_Movies: ['No' 'Yes' 'No internet service']

-----
Contract: ['Month-to-month' 'One year' 'Two year']

-----
Paperless_Billing: ['Yes' 'No']

-----
Payment_Method: ['Electronic check' 'Mailed check' 'Bank transfer (automatic)'
'Credit card (automatic)']

-----
Monthly_Charges: [29.85 56.95 53.85 ... 63.1  44.2  78.7 ]

-----
Total_Charges: ['29.85' '1889.5' '108.15' ... '346.45' '306.6' '6844.5']

-----
Churn: ['No' 'Yes']
```

```
1
2 # Change the data type from column into integer
3
4 telcom['Senior_Citizen'] = telcom['Senior_Citizen'].astype(int)
5
```

```
1
2 # Making This columns into a separate label column with yes and no for easy understanding in chart.
3
4 telcom['Senior_Citizen_Status'] = telcom["Senior_Citizen"].map({0:"No", 1:"Yes"})
5 telcom["Churn_Flag"] = telcom['Churn'].map({"Yes":1, "No":0})
```

```
1
2 telcom.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 23 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Customer_Id      7043 non-null   object
1   Gender           7043 non-null   object
2   Senior_Citizen   7043 non-null   int64
3   Partner          7043 non-null   object
4   Dependents       7043 non-null   object
5   Tenure           7043 non-null   int64
6   Phone_Service    7043 non-null   object
7   Multiple_Lines   7043 non-null   object
8   Internet_Service 7043 non-null   object
9   Online_Security  7043 non-null   object
10  Online_Backup     7043 non-null   object
11  Device_Protection 7043 non-null   object
12  Tech_Support      7043 non-null   object
13  Streaming_Tv      7043 non-null   object
14  Streaming_Movies  7043 non-null   object
15  Contract          7043 non-null   object
16  Paperless_Billing 7043 non-null   object
17  Payment_Method    7043 non-null   object
18  Monthly_Charges   7043 non-null   float64
19  Total_Charges     7043 non-null   object
20  Churn             7043 non-null   object
21  Senior_Citizen_Status 7043 non-null   object
22  Churn_Flag        7043 non-null   int64
dtypes: float64(1), int64(3), object(19)
memory usage: 1.2+ MB
```

```
1
2 telcom['Total_Charges'].astype(float) # There is some blank space
```

```
-----
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_647/1855022703.py in <cell line: 0>()
----> 1 telcom['Total_Charges'].astype(float) # There is some blank space

-----
6 frames
/usr/local/lib/python3.13/dist-packages/pandas/core/dtypes/astype.py in _astype_nansafe(arr, dtype, copy, skipna)
    131     if copy or arr.dtype == object or dtype == object:
    132         # Explicit copy, or required since NumPy can't view from / to object.
--> 133         return arr.astype(dtype, copy=True)
    134
    135     return arr.astype(dtype, copy=copy)

ValueError: could not convert string to float: ' '
```

```
1
2 telcom[telcom['Total_Charges']== " "]
```

	Customer_Id	Gender	Senior_Citizen	Partner	Dependents	Tenure	Phone_Service	Multiple_Lines	Internet_Service	Online_Security	...	Streaming_Tv	Streaming_Movies	Contract	Pape
488	4472-LVYGI	Female	0	Yes	Yes	0	No	No phone service	DSL	Yes	...	Yes	No	Two year	
753	3115-CZMZD	Male	0	No	Yes	0	Yes	No	No	No internet service	...	No internet service	No internet service	Two year	
936	5709-LVOEQ	Female	0	Yes	Yes	0	Yes	No	DSL	Yes	...	Yes	Yes	Two year	
1082	4367-NUYAO	Male	0	Yes	Yes	0	Yes	Yes	No	No internet service	...	No internet service	No internet service	Two year	
1340	1371-DWPAZ	Female	0	Yes	Yes	0	No	No phone service	DSL	Yes	...	Yes	No	Two year	
3331	7644-OMVMY	Male	0	Yes	Yes	0	Yes	No	No	No internet service	...	No internet service	No internet service	Two year	
3826	3213-VVOLG	Male	0	Yes	Yes	0	Yes	Yes	No	No internet service	...	No internet service	No internet service	Two year	
4380	2520-SGTTA	Female	0	Yes	Yes	0	Yes	No	No	No internet service	...	No internet service	No internet service	Two year	
5218	2923-ARZLG	Male	0	Yes	Yes	0	Yes	No	No	No internet service	...	No internet service	No internet service	One year	
6670	4075-WKNIU	Female	0	Yes	Yes	0	Yes	Yes	DSL	No	...	Yes	No	Two year	
6754	2775-SEFEE	Male	0	No	Yes	0	Yes	Yes	DSL	Yes	...	No	No	Two year	

11 rows × 23 columns

```
1
2 ## Droping those blank space total chrages because they are in very less number
3 telcom = telcom[telcom['Total_Charges'] != " "].reset_index(drop = True)
4 telcom['Total_Charges'] = telcom['Total_Charges'].astype(float)
```

```
1
2 # Clean Data
3 df = telcom
```

```
1
2 df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7032 entries, 0 to 7031
Data columns (total 23 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Customer_Id            7032 non-null   object
1   Gender                 7032 non-null   object
2   Senior_Citizen         7032 non-null   int64
3   Partner               7032 non-null   object
4   Dependents            7032 non-null   object
5   Tenure                7032 non-null   int64
6   Phone_Service         7032 non-null   object
7   Multiple_Lines        7032 non-null   object
8   Internet_Service      7032 non-null   object
9   Online_Security       7032 non-null   object
10  Online_Backup         7032 non-null   object
11  Device_Protection     7032 non-null   object
12  Tech_Support          7032 non-null   object
13  Streaming_Tv          7032 non-null   object
14  Streaming_Movies      7032 non-null   object
15  Contract              7032 non-null   object
16  Paperless_Billing     7032 non-null   object
17  Payment_Method        7032 non-null   object
18  Monthly_Charges       7032 non-null   float64
19  Total_Charges         7032 non-null   float64
20  Churn                 7032 non-null   object
21  Senior_Citizen_Status 7032 non-null   object
22  Churn_Flag            7032 non-null   int64
dtypes: float64(2), int64(3), object(18)
memory usage: 1.2+ MB
```

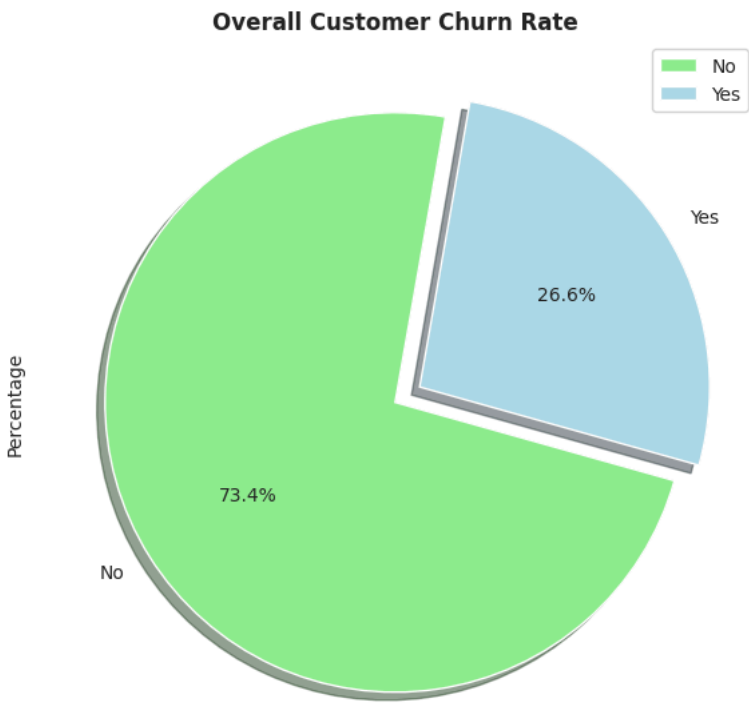
```
1
2 df.head()
```

	Customer_Id	Gender	Senior_Citizen	Partner	Dependents	Tenure	Phone_Service	Multiple_Lines	Internet_Service	Online_Security	...	Streaming_Tv	Streaming_Movies	Contract	Paperless
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	No	...	No	No	Month-to-month	
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes	...	No	No	One year	
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Yes	...	No	No	Month-to-month	
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes	...	No	No	One year	
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No	...	No	No	Month-to-month	

5 rows × 23 columns

"Overall Customer Churn Rate"

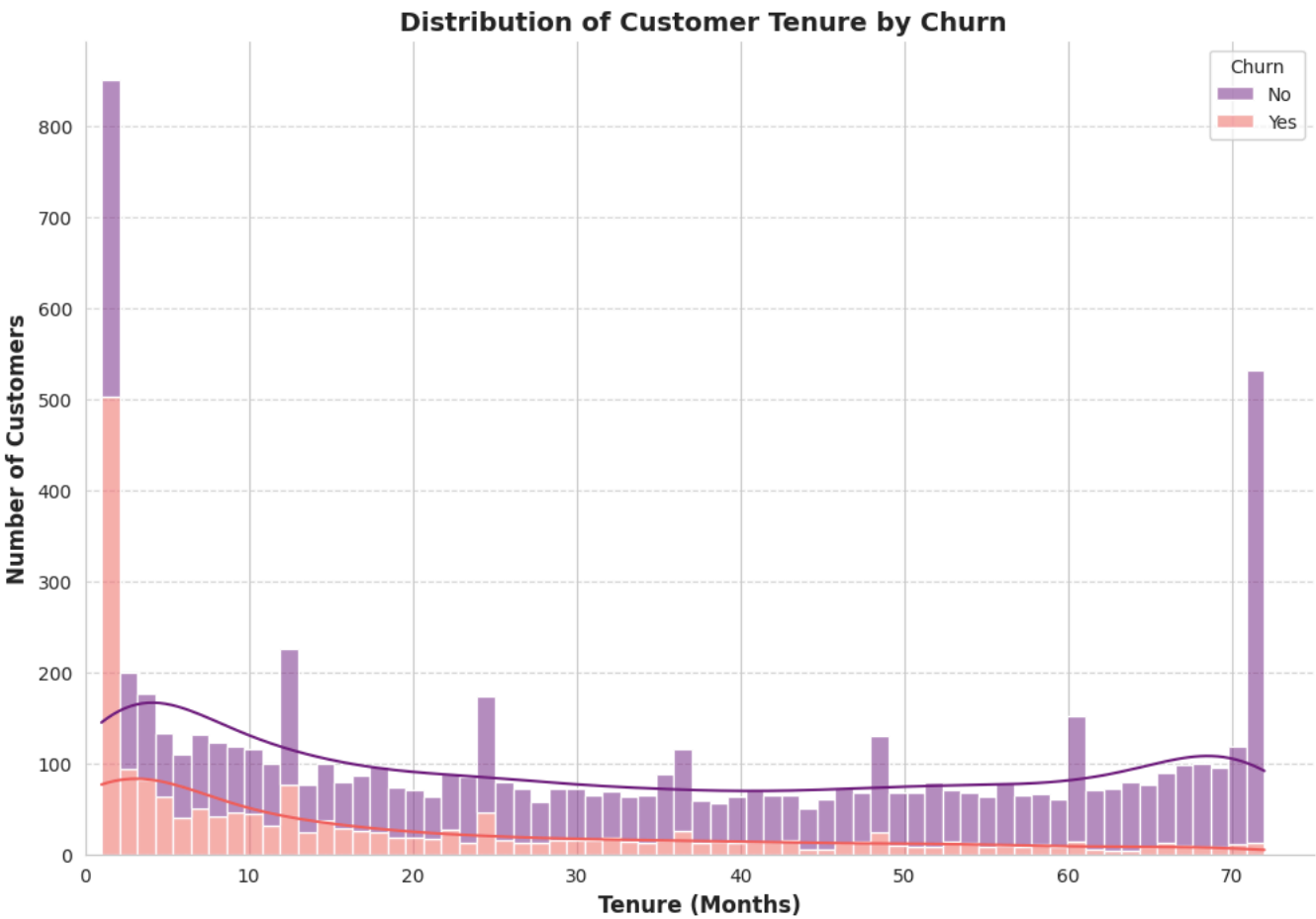
```
1
2 df['Churn'].value_counts().plot.pie(figsize=(7,8),autopct="%1.1f%%",
3                                     colors= ["lightgreen","lightblue"], explode = (0,0.1),startangle = 80, shadow = True)
4
5 plt.title("Overall Customer Churn Rate",fontsize = 12,fontweight="bold")
6 plt.legend(labels = ["No","Yes"])
7 plt.ylabel("Percentage")
8 plt.show()
```



1. The Retention Cliff: Analyzing Customer Attrition by Tenure

Getting a new customer costs time and marketing money. If a customer leaves too quickly, the company loses money on that signup. In this section, we analyze customer Tenure (how many months they stay) against Churn to see exactly when most customers decide to leave.

```
1
2 plt.figure(figsize=(12,8))
3
4 # Plot the distribution with a Kernel Density Estimate (KDE) curve
5 sns.histplot(data=df, x='Tenure', hue='Churn', multiple='stack',
6             kde=True, bins=65, palette="magma")
7
8 plt.title(' Distribution of Customer Tenure by Churn', fontsize=14, fontweight='bold')
9 plt.xlabel('Tenure (Months)', fontsize=12, fontweight="bold")
10 plt.ylabel('Number of Customers', fontsize=12, fontweight="bold")
11 plt.xlim(0, 75)
12 plt.grid(axis='y', linestyle='--', alpha=0.7)
13 sns.despine()
14
15
16 plt.show()
```



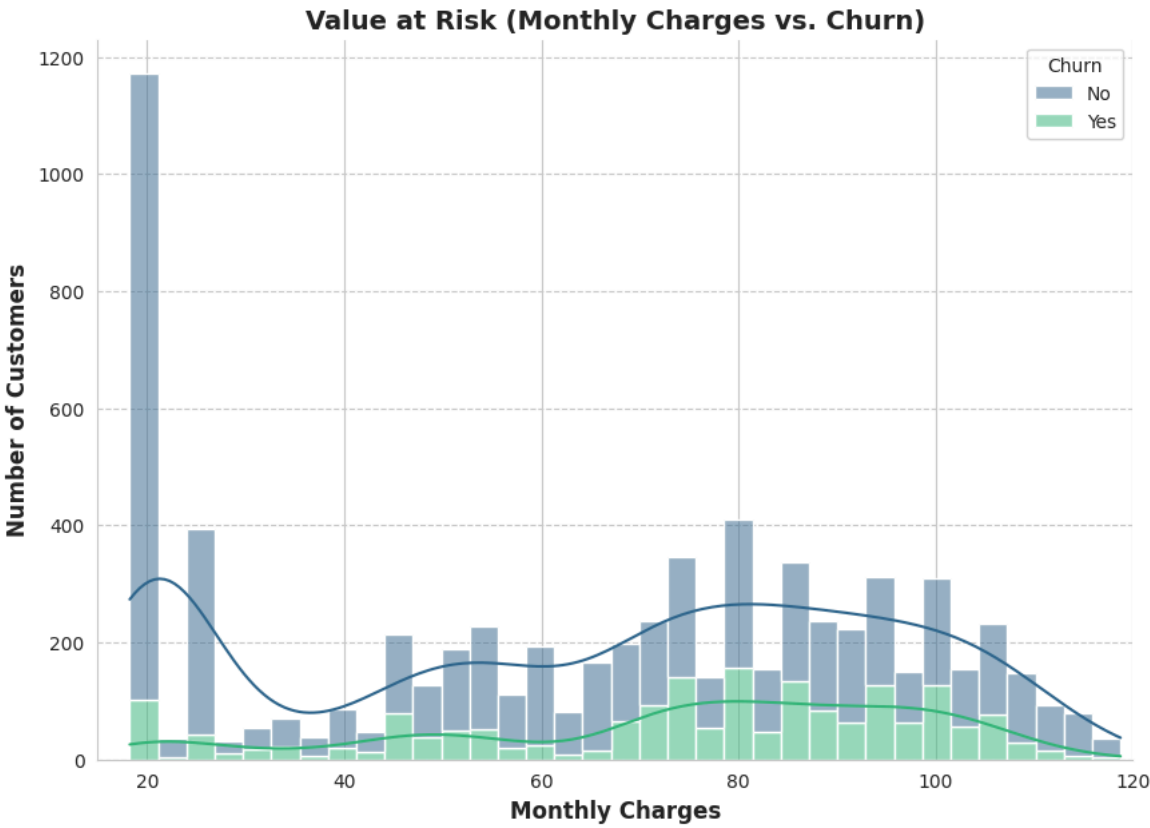
Insights

- **Early Drop-off:** Churn is highest in the first 1 to 5 months. Many customers leave almost immediately after joining.
- **Long-Term Loyalty:** Customers who stay past their first year (12+ months) are far less likely to leave and become very loyal.
- **Business Recommendation:** The company needs to improve the new customer onboarding experience and offer early support or check-ins during the first 90 to 120 days to prevent early cancellations.

2. Monthly Charges vs. Churn

Problem Statement: Not every lost customer impacts the company equally. Losing a customer on a cheap plan hurts less than losing a customer paying a premium bill every month. Here, we analyze Monthly Charges against Churn to see whether budget customers or high-paying customers are leaving more often.

```
1
2 # Histogram
3
4 plt.figure(figsize=(10,7))
5
6 sns.histplot(data = df , x= "Monthly_Charges",kde = True, hue="Churn",multiple = "stack",bins = 35,palette = "viridis")
7
8 plt.title("Value at Risk (Monthly Charges vs. Churn)",fontsize = 14,fontweight = "bold")
9 plt.xlabel("Monthly Charges",fontsize = 12,fontweight = "bold")
10 plt.ylabel("Number of Customers",fontsize = 12,fontweight = "bold")
11 plt.grid(axis = 'y',linestyle = "--")
12 sns.despine()
13 plt.xlim(15,120)
14 plt.show()
15
```



```
1
2 # 1. Filter the dataset to ONLY include people paying between $20 and $30
3 budget_tier = telcom[(telcom['Monthly_Charges'] >= 20) & (telcom['Monthly_Charges'] <= 30)]
4
5 # 2. Calculate the percentage of churn in this specific group
6 budget_churn_rate = budget_tier['Churn_Flag'].mean() * 100
7
8 print(f"Exact Churn Rate for $20-$30 range: {budget_churn_rate:.2f}%")
9
10
11 # 1. Filter the dataset to ONLY include people paying between $20 and $30
12 budget_tier = telcom[(telcom['Monthly_Charges'] >=75 ) & (telcom['Monthly_Charges'] <= 120)]
13
14 # 2. Calculate the exact percentage of churn in this specific group
15 budget_churn_rate = budget_tier['Churn_Flag'].mean() * 100
16
17 print(f"Exact Churn Rate for $75-$120 range: {budget_churn_rate:.2f}%")
```

Exact Churn Rate for \$20-\$30 range: 10.33%
Exact Churn Rate for \$75-\$120 range: 34.67%

Insights

- **Budget Customers are Stable:** Customers paying low monthly bills (20–30) have a very low churn rate of **10.33%** and stay loyal.
- **High-Bill Risk:** Churn increases to **34.67%** for customers with higher monthly charges (75–120).
- **Business Recommendation:** High-paying customers might feel they are not getting enough value for the price. The company should review the quality of its premium plans or bundle helpful services to make high bills feel worth the cost.

2.2 Follow-up Investigation: The "Trapped" Customer Persona (Demographics vs. Price Sensitivity)

In Question 2, we established a clear rule: high monthly charges mean high churn. But in the real world, human behavior is messy — does that rule apply to everyone?

The effort it takes to switch internet or phone providers matters. If a customer is alone, switching is easy. But with a family relying on multiple devices, streaming accounts, and work-from-home setups, switching providers is a much bigger operational headache.

Questions:

- Do customers without dependents churn rapidly when bills get high?
- Do customers with dependents tolerate higher bills simply because the friction of switching is too high?

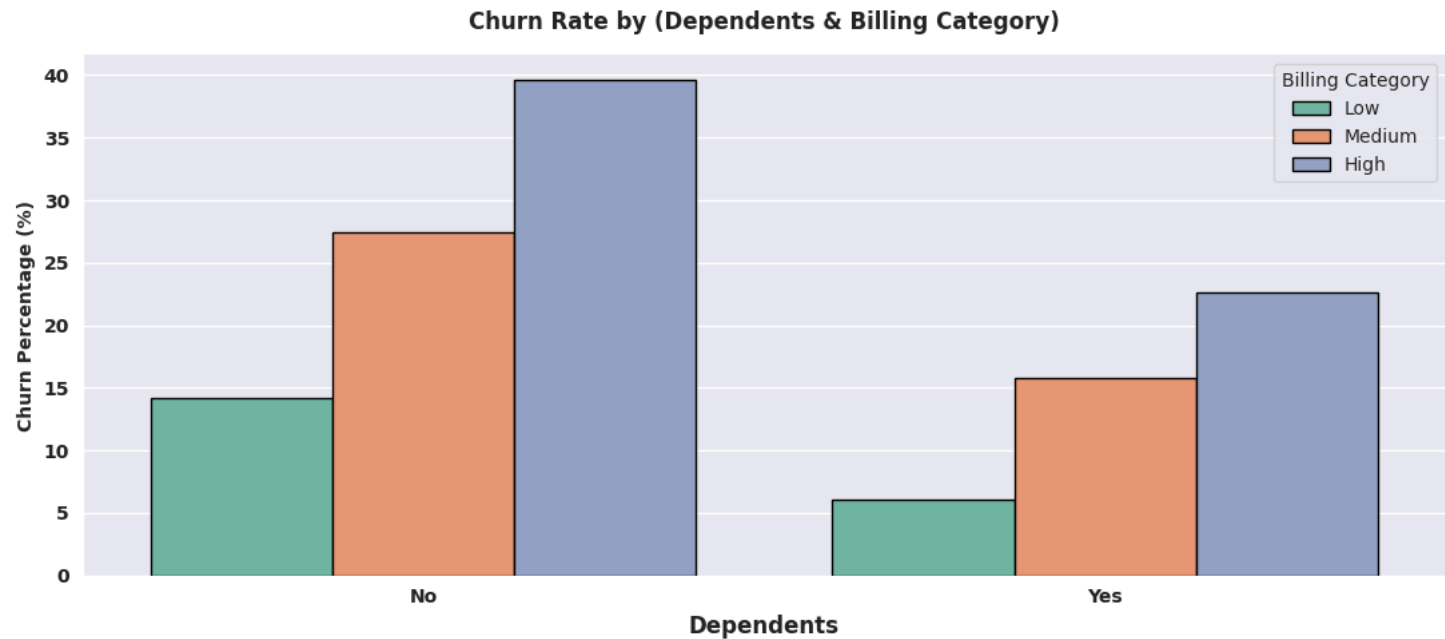
```
1
2 df["Billing_Category"] = pd.cut(df['Monthly_Charges'],
3                               bins = [0,35,70,120],
4                               labels = ['Low', 'Medium', 'High']
5                               )
6
7
8 Dependent_Churn = df.groupby(['Dependents', "Billing_Category"])[ "Churn_Flag"].mean().reset_index()
9
10 # Change it to percentage
```

```
11
12 Dependent_Churn["Churn_Percentage"] = Dependent_Churn["Churn_Flag"]*100
13
14 Dependent_Churn
15
16
17
18
```

/tmp/ipykernel_647/2753988380.py:7: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior.
Dependent_Churn = df.groupby(['Dependents', 'Billing_Category'])['Churn_Flag'].mean().reset_index()

Dependents	Billing_Category	Churn_Flag	Churn_Percentage
0	No	Low	0.141482
1	No	Medium	0.274607
2	No	High	0.396276
3	Yes	Low	0.060870
4	Yes	Medium	0.157895
5	Yes	High	0.226562

```
1
2 # Making The Plot
3
4
5 plt.figure(figsize=(11,5))
6 sns.set_style("darkgrid")
7
8
9 sns.barplot(data = Dependent_Churn , x='Dependents', y='Churn_Percentage',
10             hue='Billing_Category', palette='Set2', edgecolor = "black")
11
12 plt.title('Churn Rate by (Dependents & Billing Category)', fontsize=12, fontweight='bold', pad=14)
13 plt.ylabel('Churn Percentage (%)', fontsize=10, fontweight='bold')
14 plt.xlabel('Dependents', fontsize=12, fontweight='bold')
15
16 plt.xticks(fontweight = "bold")
17 plt.yticks(fontweight = "bold")
18
19 plt.legend(title = "Billing Category")
20 sns.despine()
21 plt.tight_layout()
22 plt.show()
23
24
```



Insights

This first look suggests customers without dependents churn faster as bills increase — but Dependents alone is an incomplete measure of household complexity, since it ignores whether the customer has a partner.

To get a clearer picture, Partner and Dependents are combined into a single Household_Type (Alone / Couple / Family), and Add-On Count is also checked, since more add-ons could independently explain lower churn regardless of household type.

```
1
2 # Making a new column that shows the count of add ons customers have
3
4 add_on_col = ['Online_Security', 'Online_Backup', 'Device_Protection', 'Tech_Support',
5              'Streaming_Tv', 'Streaming_Movies']
6
7
8 df['Add_On_Count'] = (df[add_on_col]=='Yes').sum(axis =1)
```

```
1
2 # Making a Household type col to flag
3
4 # No partner + No dependents → Alone
5 # Partner + No dependents → Couple
6 # Any dependents (regardless of partner) → Family
7
8 conditions = [
9     df['Dependents'] == 'Yes',
10     (df['Partner'] == 'Yes') & (df['Dependents'] == 'No'),
11     (df['Partner'] == 'No') & (df['Dependents'] == 'No')
12 ]
13
14 choices = ['Family', 'Couple', 'Alone']
15
16 df['Household_Type'] = np.select(conditions, choices, default='Unknown') # Added a default for completeness
17
18 # Display the value counts for the new column
19 df['Household_Type'].value_counts()
```

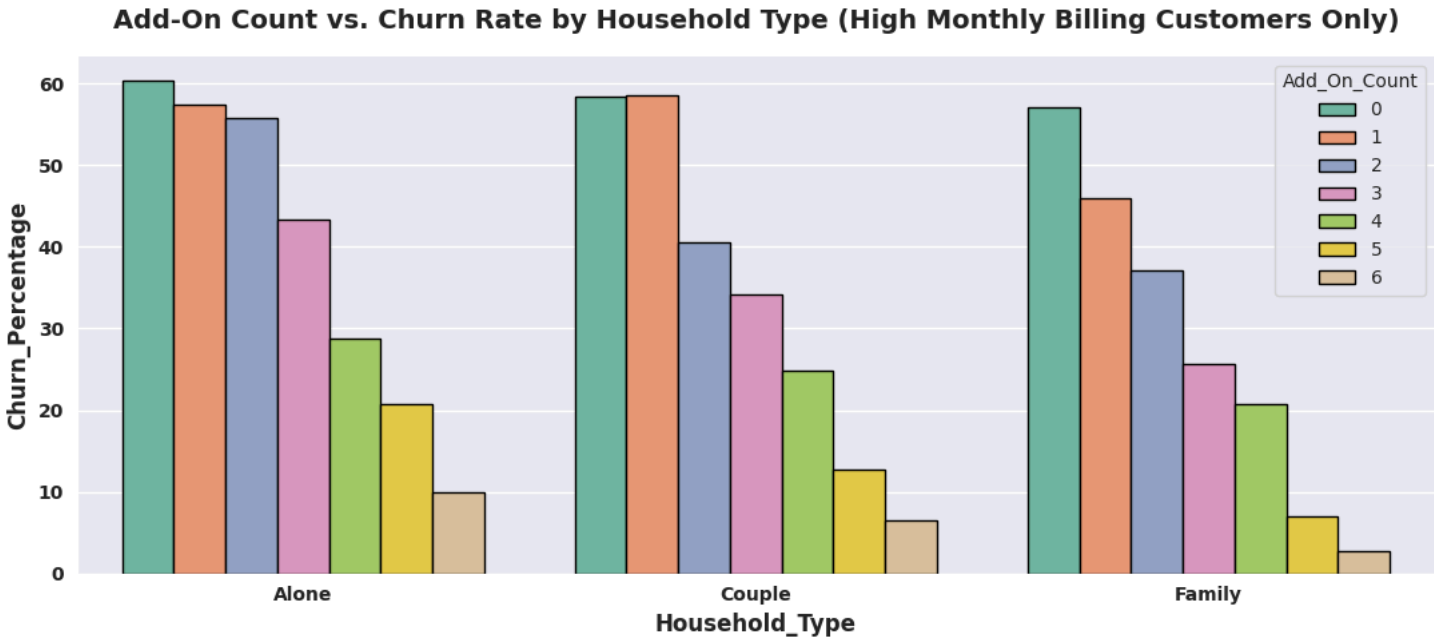
count	
Household_Type	
Alone	3280
Family	2099
Couple	1653

dtype: int64

```
1
2 churn_rate_high_billing = df[df['Billing_Category'].isin(['High'])] \
3 .groupby(['Add_On_Count', 'Household_Type'])['Churn_Flag'].mean().reset_index()
4
5
6 churn_rate_high_billing['Churn_Percentage'] = churn_rate_high_billing['Churn_Flag'] * 100
7
```

```
1
2 def plot(data,x,y,hue,title,x_label,y_label,legend):
3
4     plt.figure(figsize = (11,5))
5     sns.set_style("darkgrid")
6
7     sns.barplot(data = data, x = x, y = y ,hue = hue, palette = "Set2", edgecolor = "black")
8
9     plt.title(title,fontweight = "bold",fontsize = 14, pad = 15)
10    plt.xlabel(x_label,fontweight = "bold",fontsize = 12)
11    plt.ylabel(y_label,fontweight = "bold",fontsize = 12)
12
13    plt.xticks(fontweight = "bold")
14    plt.yticks(fontweight = "bold")
15
16    plt.legend(title = legend)
17    sns.despine()
18
19    plt.tight_layout()
20    plt.show()
```

```
1
2 plot(churn_rate_high_billing , "Household_Type", "Churn_Percentage", "Add_On_Count",
3      "Add-On Count vs. Churn Rate by Household Type (High Monthly Billing Customers Only)",
4      "Household_Type", "Churn_Percentage", "Add_On_Count",)
```



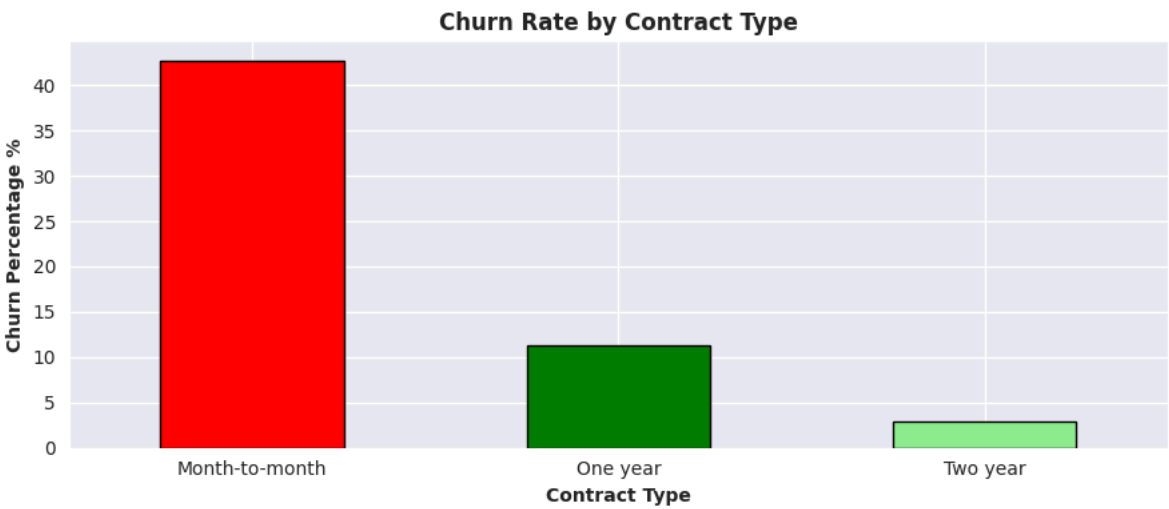
Insights

- **Finding:** Among High-billing customers, churn drops sharply as the number of add-ons increases — from about 57–60% churn with 0 add-ons down to just 3–10% churn with 6 add-ons. This same pattern holds whether the customer is Alone, a Couple, or a Family — household type doesn't meaningfully change the story.
- **So What:** How many add-ons a customer has matters much more than their household type. The real risk group is High-billing customers with few or no add-ons — not "singles" or "families" specifically. *(Note: Add-On Count and Tenure showed a moderate positive correlation of 0.5, so this should be read as a strong association rather than a fully proven independent cause.)*
- **Recommendation:** The company should offer discounted add-on bundles (e.g. Tech Support + Security) to High-billing customers with 0–2 add-ons. This group shows the highest churn risk and the clearest opportunity to reduce it.

3. Churn Rate by Contract Type (Month-to-month vs One year vs Two year)

Does contract length influence customer churn, and if so, by how much? Understanding this relationship helps the company evaluate whether encouraging longer-term contracts could be an effective retention strategy.

```
1
2 churn_rate_by_contract = df.groupby("Contract")['Churn_Flag'].mean()*100
3
4 plt.figure(figsize=(9,4))
5 churn_rate_by_contract.plot(kind="bar", color=["red","green","lightgreen"], edgecolor="black")
6 plt.title("Churn Rate by Contract Type", fontsize=12, fontweight="bold")
7 plt.xlabel("Contract Type", fontsize=10, fontweight="bold")
8 plt.ylabel("Churn Percentage %", fontsize=10, fontweight="bold")
9 plt.xticks(rotation=0) # keep labels horizontal, easier to read
10
11
12 plt.tight_layout()
13 plt.show()
```



Insights

- **Finding:** Month-to-month churn is above ~40%, compared to ~10% for one-year and ~3% for two-year contracts. Contract type shows the largest churn gap of any factor tested so far — this needs further investigation before drawing conclusions about *why*.
- **Business Recommendation:** The company should still prioritize converting month-to-month customers to longer-term contracts through discounts or added perks.

3.1 Follow-up Investigation: Does Payment Method Add Further Risk on Top of Contract Type?

We already found that contract type and internet service both affect churn on their own, and that missing Tech Support explains part of why Fiber optic churns so much.

Now the question is: does combining contract type with payment method reveal an even bigger risk group than either factor alone?

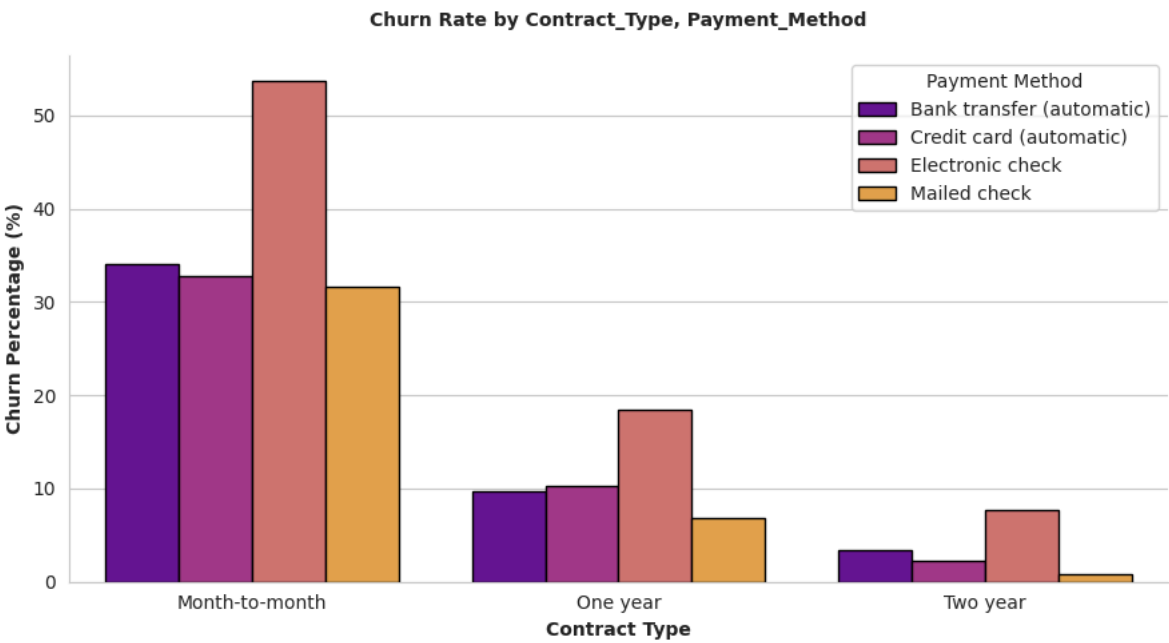
Knowing "month-to-month customers churn more" isn't very useful by itself — it's too broad. But if one specific combination shows a churn rate far higher than the rest, that gives the company an exact group to target for retention.

```
1
2 #Making a group by to see Tenure and Churn Rate for contract type and payment_method
3
4 plot_data = df.groupby(["Contract", "Payment_Method"])[["Tenure", "Churn_Flag"]].mean().reset_index()
5
6 plot_data["Churn_Percentage"] = plot_data['Churn_Flag']*100
```

```
1
2 plot_data
```

	Contract	Payment_Method	Tenure	Churn_Flag	Churn_Percentage
0	Month-to-month	Bank transfer (automatic)	24.904924	0.341256	34.125637
1	Month-to-month	Credit card (automatic)	23.955801	0.327808	32.780847
2	Month-to-month	Electronic check	17.971351	0.537297	53.729730
3	Month-to-month	Mailed check	10.042553	0.315789	31.578947
4	One year	Bank transfer (automatic)	46.017903	0.097187	9.718670
5	One year	Credit card (automatic)	45.477387	0.103015	10.301508
6	One year	Electronic check	46.178674	0.184438	18.443804
7	One year	Mailed check	29.211310	0.068452	6.845238
8	Two year	Bank transfer (automatic)	61.822064	0.033808	3.380783
9	Two year	Credit card (automatic)	59.910345	0.022414	2.241379
10	Two year	Electronic check	61.113095	0.077381	7.738095
11	Two year	Mailed check	43.752000	0.008000	0.800000

```
1
2 plt.figure(figsize=(9,5))
3 sns.set_style("whitegrid")
4
5
6 sns.barplot(data = plot_data, x='Contract', y='Churn_Percentage', hue='Payment_Method', palette='plasma', edgecolor='black')
7
8 plt.title('Churn Rate by Contract_Type, Payment_Method', fontsize=10, fontweight='bold', pad=16)
9 plt.ylabel('Churn Percentage (%)', fontsize=10, fontweight='bold')
10 plt.xlabel('Contract Type', fontsize=10, fontweight='bold')
11
12 plt.legend(title = "Payment Method")
13 sns.despine()
14
15 plt.tight_layout()
16 plt.show()
```



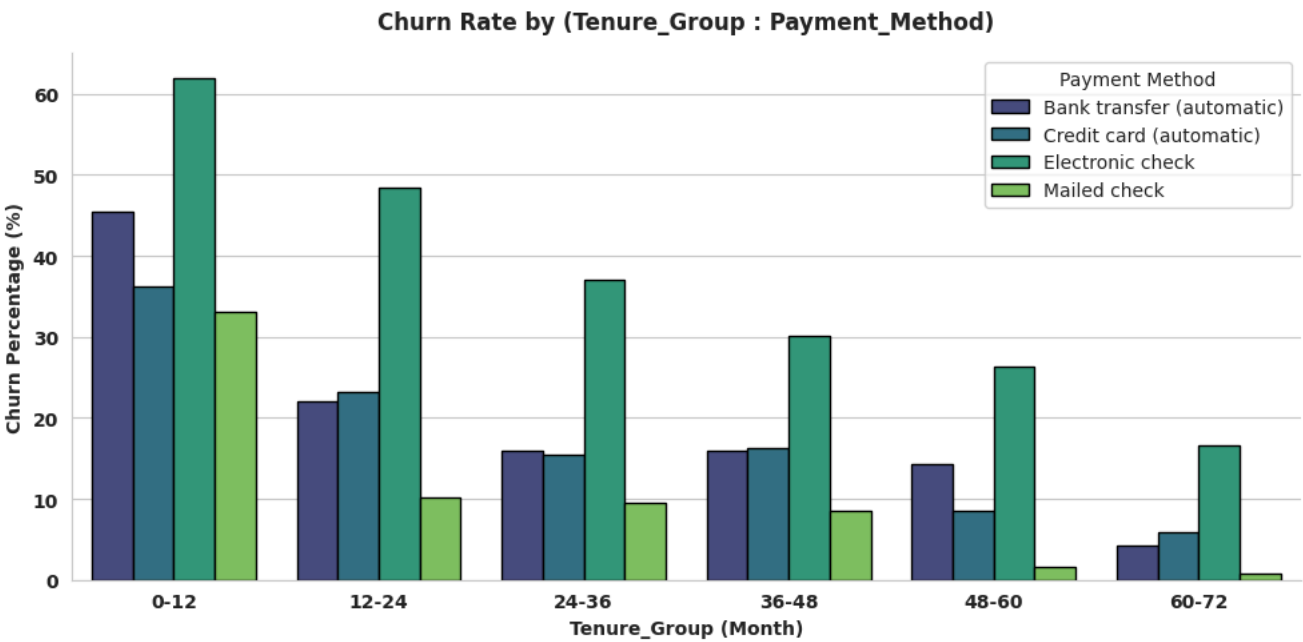
Insights

The table shows Electronic check has the highest churn in every contract type, but those customers also tend to have lower average tenure. This makes it unclear whether Electronic check itself is risky, or whether it just looks risky because those customers are newer. To check this, customers were grouped into tenure ranges (in months), and churn rate was compared by payment method within each range.

```
1
2 # Create tenure beans
3
4 df["Tenure_Group"] = pd.cut(df['Tenure'],
5                             bins = [0,12,24,36,48,60,72],
6                             labels = ["0-12", "12-24", "24-36", "36-48", "48-60", "60-72"]
7                             )
8 t_group = df.groupby(["Tenure_Group", "Payment_Method"])[["Churn_Flag"]].mean().reset_index()
9
10 # Make it to the percentage
11
12 t_group["Churn_Percentage"] = t_group["Churn_Flag"]*100
```

/tmp/ipykernel_647/251089048.py:7: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior.
t_group = df.groupby(["Tenure_Group", "Payment_Method"])[["Churn_Flag"]].mean().reset_index()

```
1
2 plt.figure(figsize=(10,5))
3 sns.set_style("whitegrid")
4
5 sns.barplot(data = t_group, x='Tenure_Group', y='Churn_Percentage', hue='Payment_Method', palette='viridis', edgecolor = "black")
6
7 plt.title('Churn Rate by (Tenure_Group : Payment_Method)', fontsize=12, fontweight='bold', pad=13)
8 plt.ylabel('Churn Percentage (%)', fontsize=10, fontweight='bold')
9 plt.xlabel('Tenure_Group (Month)', fontsize=10, fontweight='bold')
10
11 plt.xticks(fontweight = "bold")
12 plt.yticks(fontweight = "bold")
13
14 plt.legend(title = "Payment Method")
15 sns.despine()
16
17 plt.tight_layout()
18 plt.show()
```



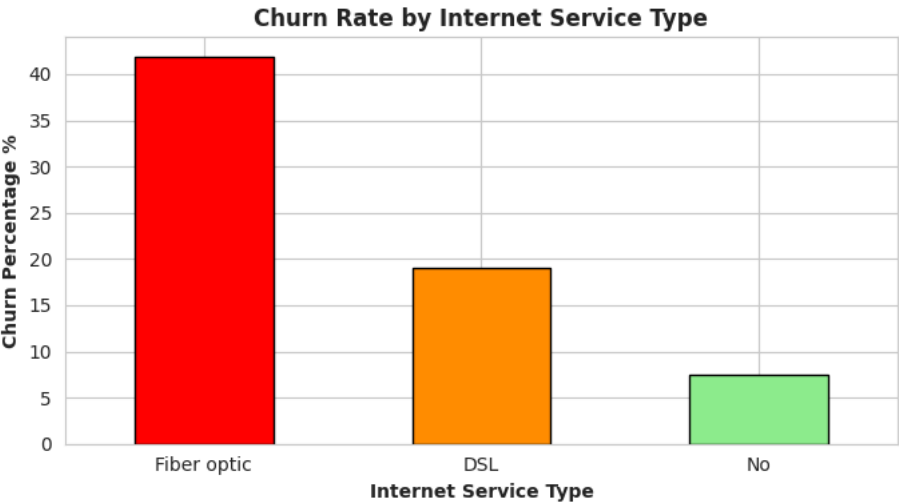
Insights

- **Finding:** Across every tenure range, Electronic check consistently has the highest churn rate of all payment methods — confirming it's a real, independent risk factor and not just a byproduct of those customers being newer.
- **So What:** Switching a customer's payment method away from Electronic check could reduce churn risk on its own, regardless of how long they've been a customer or what contract they're on.
- **Recommendation:** The company should actively encourage Electronic check users to switch to automatic bank transfer or credit card payment — for example, through a small discount or incentive — since this is a targetable, low-effort lever that applies across the entire customer base, not just one segment.

4. Internet Service Type vs Churn

Problem Statement: Does the type of internet service (DSL, Fiber optic, or none) affect churn rate? This helps determine whether the company's premium Fiber offering is retaining customers as expected, or losing them faster than lower-tier services.

```
1
2 # Calculating The Churn Rratio
3
4 internet_service_churn = df.groupby("Internet_Service")["Churn_Flag"].mean().sort_values(ascending = False)*100
5
6
7 plt.figure(figsize=(7,4))
8
9 internet_service_churn.plot(kind="bar", color= ['red','darkorange',"lightgreen"], edgecolor = "black")
10 plt.title("Churn Rate by Internet Service Type", fontsize = 12, fontweight = "bold")
11 plt.xlabel("Internet Service Type", fontsize = 10, fontweight = "bold")
12 plt.ylabel("Churn Percentage %", fontsize = 10, fontweight = "bold")
13 plt.xticks(rotation = 0)
14
15 plt.tight_layout()
16 plt.show()
17
18
```



Insights

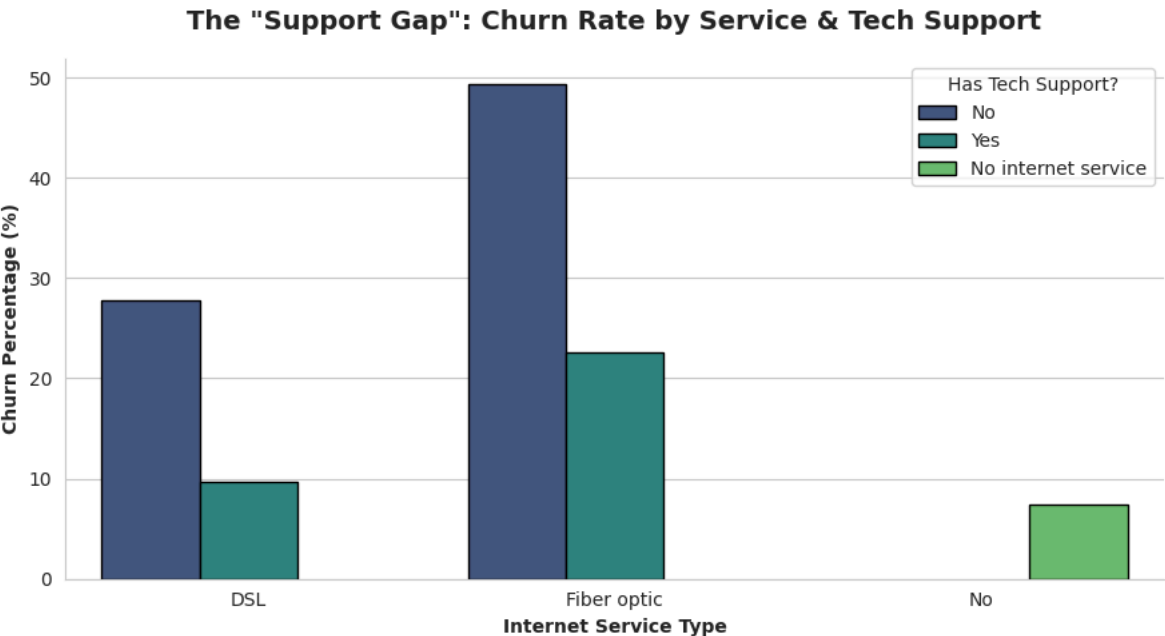
Fiber optic customers churn at 40%+, more than double DSL's 20%, and far higher than customers with no internet service (~5%).

4.1 Follow-up Investigation: Is Fiber Optic's High Churn Really About the Product, or Explained by Something Else?

Fiber optic customers showed ~40%+ churn vs DSL's ~20% — but this could be a genuine service/support gap (Fiber customers lacking Tech Support more often).

Testing: churn rate for Fiber optic customers with vs. without Tech Support, to see if support presence changes the picture.

```
1
2 plt.figure(figsize=(9, 5))
3 sns.set_style("whitegrid")
4
5 # Calculate the churn rate data
6 plot_data = df.groupby(['Internet_Service', 'Tech_Support'])['Churn_Flag'].mean().reset_index()
7 plot_data['Churn_Rate_%'] = plot_data['Churn_Flag'] * 100
8
9 sns.barplot(data=plot_data, x='Internet_Service', y='Churn_Rate_%', hue='Tech_Support', palette='viridis', edgecolor='black')
10
11 plt.title('The "Support Gap": Churn Rate by Service & Tech Support', fontsize=14, fontweight='bold', pad=16)
12 plt.ylabel('Churn Percentage (%)', fontsize=10, fontweight='bold')
13 plt.xlabel('Internet Service Type', fontsize=10, fontweight='bold')
14
15 plt.legend(title='Has Tech Support?')
16 sns.despine()
17
18 plt.tight_layout()
19 plt.show()
```



Insights

- **Finding:** Within Fiber optic, customers without Tech Support churn at ~50%, versus ~23% for those with Tech Support — more than double. The same pattern holds for DSL (28% vs 10%), but Fiber's gap is by far the largest in absolute terms.
- **So What:** Fiber optic customers without Tech Support are the single highest-risk segment in the dataset (~50% churn) — nearly 1 in 2 leave. This suggests the "premium" Fiber product is genuinely underperforming for customers who aren't also paying for support — likely because Fiber has more complexity or reliability issues that unsupported customers can't resolve on their own.

- **Recommendation:** The company should prioritize offering Tech Support (potentially bundled or discounted) specifically to Fiber optic customers, since this segment shows the largest churn-reduction opportunity of any factor tested — cutting churn roughly in half within this group alone.

4.2 Follow-up Investigation: Does Tech Support Matter More for Senior Citizens?

Problem Statement: Tech Support was already shown to reduce churn overall (the "Toxic Product" finding). The next question is whether this effect is the same for everyone, or whether some groups benefit more from Tech Support than others — specifically, do Senior Citizens without Tech Support churn even more than non-seniors without Tech Support?

```
1
2 # Make a new column that shows senior  as yes and no easy to understand
3
4 df['Senior_Citizen_Status'] = np.where(df["Senior_Citizen"] == 1, "Yes", "No")
5
```

```
1
2  Senior_Churn = df.groupby(["Senior_Citizen_Status", "Tech_Support"])["Churn_Flag"].mean().reset_index()
3
4  Senior_Churn["Churn_Percentage"] = Senior_Churn["Churn_Flag"]*100
5
6  Senior_Churn
```

	Senior_Citizen_Status	Tech_Support	Churn_Flag	Churn_Percentage
0	No	No	0.388342	38.834217
1	No	No internet service	0.073569	7.356948
2	No	Yes	0.145506	14.550562
3	Yes	No	0.506024	50.602410
4	Yes	No internet service	0.096154	9.615385
5	Yes	Yes	0.196154	19.615385

```
1
2 plot(Senior_Churn,"Senior_Citizen_Status", "Churn_Percentage", "Tech_Support", "Senior_Churn for Tech_Support",
3      'Senior_Citizen_Status', 'Churn_Percentage', "Tech_Support")
```

